





Jeux video IA

Chapitre 04 : Introduction aux algorithmes génétiques

Focus du chapitre:

Ce chapitre présente les algorithmes génétiques comme une méthode d'optimisation inspirée de l'évolution naturelle pour rechercher de bonnes solutions dans de grands espaces complexes.

Pr. Youness BOUTYOUR

ENSIAS · Université Mohammed V

20 avr 2026

Semestre 2 | Année 2025-2026

Plan du chapitre

I

Fondements

II

Algorithme génétique canonique

III

Représentations : binaire, réelle, permutation, arbre, contraintes

IV

Opérateurs génétiques

V

Mini-cases

Jeux Vidéo IA

Introduction aux algorithmes génétiques

3/76

Objectifs pédagogiques

➤ Expliquer le principe évolutionnaire : sélection, héritage, mutation.

➤ Modéliser un problème sous forme de population de chromosomes.

➤ Calculer une fitness et effectuer une itération simple à la main.

➤ Identifier les paramètres critiques : taille, croisement, mutation, arrêt.

➤ Relier les AG aux jeux vidéo : trajectoires, comportements, génération de contenu.

Jeux Vidéo IA

Introduction aux algorithmes génétiques

2/76

I. Fondements

Motivation

Pourquoi utiliser les algorithmes génétiques?

❑ Résoudre des problèmes d'optimisation difficiles lorsque l'espace de recherche est très grand.

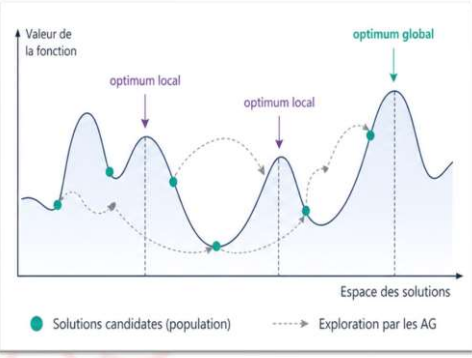
❑ Trouver de bonnes solutions sans tester toutes les combinaisons possibles.

❑ Traiter des problèmes complexes, non linéaires, discontinus ou avec plusieurs optima locaux.

❑ Ne pas dépendre du calcul du gradient ni de propriétés mathématiques fortes de la fonction objectif.

❑ Explorer plusieurs solutions en parallèle grâce à une population d'individus.

❑ Maintenir un équilibre entre l'exploitation et l'exploration



Jeux Vidéo IA

Introduction aux algorithmes génétiques

4/76

I. Fondements

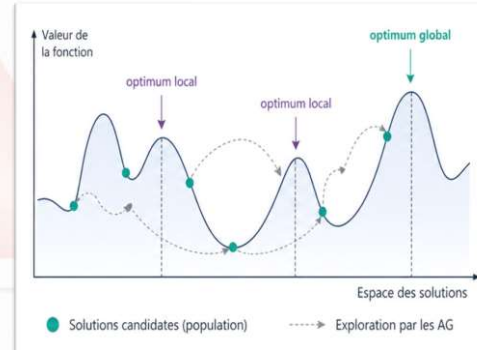
Motivation

Pourquoi utiliser les algorithmes génétiques?

❑ S'adapter à différents types de problèmes :

- optimisation combinatoire ;
- optimisation continue ;
- ordonnancement ;
- planification ;
- conception d'ingénierie ;
- jeux et intelligence artificielle.

❑ Fournir une solution approchée acceptable dans un temps raisonnable.



I. Fondements

Motivation

Ce que les AG apportent

Atouts principaux:

Les algorithmes génétiques proposent une recherche :

- populationnelle : plusieurs solutions sont explorées en parallèle ;
- stochastique : le hasard guide l'exploration sans être totalement aveugle ;
- évolutionnaire : les solutions s'améliorent progressivement génération après génération ;
- flexible : plusieurs représentations sont possibles ;
- robuste : applicable à des problèmes variés sans hypothèses mathématiques fortes.

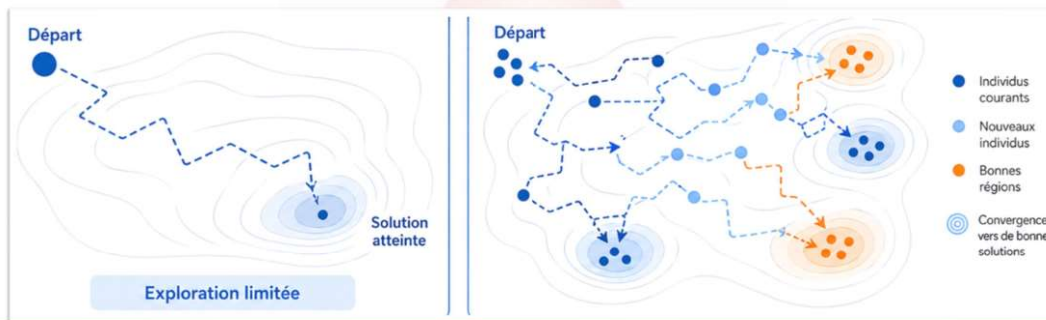
I. Fondements

Motivation

Ce que les AG apportent

Équilibre recherché

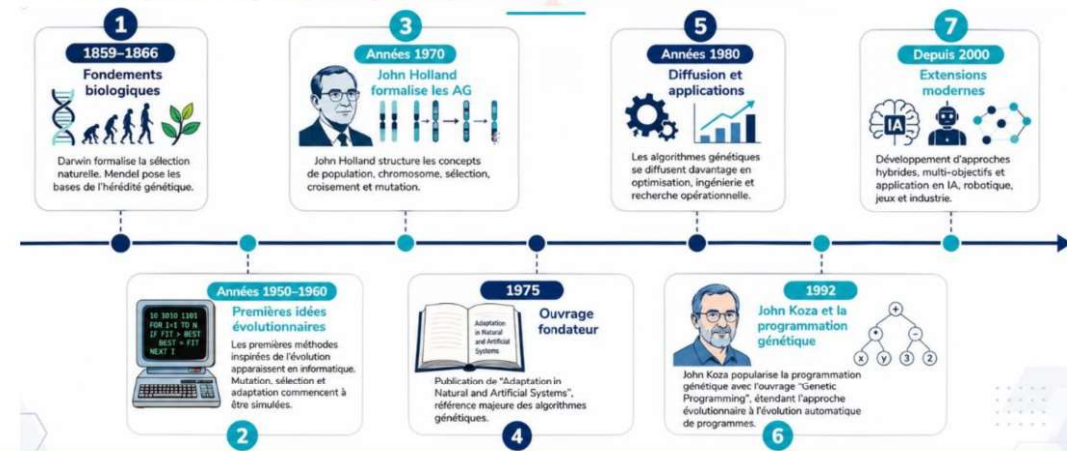
- **Exploration** : découvrir de nouvelles zones de l'espace de recherche.
- **Exploitation** : améliorer les solutions déjà prometteuses



I. Fondements

Historique

Historique des algorithmes génétiques



I. Fondements

Caractéristiques

Caractéristiques des AG

- ❑ **Représentation codée des solutions:** Les algorithmes génétiques manipulent des représentations codées des solutions, et non nécessairement les valeurs réelles des paramètres.
- ❑ **Recherche par population:** Ils travaillent sur une population de solutions candidates, plutôt que sur une solution unique.
- ❑ **Évaluation par fitness:** Ils utilisent principalement la valeur de fitness issue de la fonction objectif pour comparer les solutions.
- ❑ **Recherche probabiliste:** Ils reposent sur des mécanismes probabilistes : sélection, croisement et mutation, contrairement aux méthodes strictement déterministes.
- ❑ **Peu d'hypothèses mathématiques:** Les AG ne nécessitent pas de gradient, de dérivabilité ou de convexité.

I. Fondements

Inspiration biologique

De la biologie à l'informatique

- ❑ Une population contient des individus différents.
- ❑ Les individus mieux adaptés ont plus de chances de transmettre leurs caractéristiques.
- ❑ La recombinaison mélange l'information génétique.
- ❑ La mutation introduit de la nouveauté et maintient la diversité.



I. Fondements

Inspiration biologique

De la biologie à l'informatique

- ❑ Une population contient des individus différents.
- ❑ Les individus mieux adaptés ont plus de chances de transmettre leurs caractéristiques.
- ❑ La recombinaison mélange l'information génétique.
- ❑ La mutation introduit de la nouveauté et maintient la diversité.



Terme biologique	Correspondance informatique dans un AG
Individu	Une solution possible
Population	Ensemble de solutions candidates
Gène	Élément d'un chromosome
Chromosome	Représentation codée d'une solution
Fitness	Fonction d'évaluation de la qualité
Génération	Itération de l'algorithme

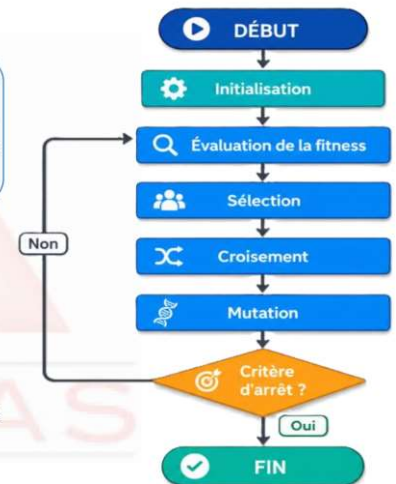
II. Algorithme génétique canonique

AG canonique

Définition

Un algorithme génétique canonique est une méthode de recherche approchée qui fait évoluer une population de solutions candidates au moyen de trois mécanismes principaux : sélection, croisement et mutation. La première version a été proposée par **John Holland en 1965**.

- Recherche par population : plusieurs candidats sont traités simultanément.
- Recherche stochastique : des tirages aléatoires guident l'exploration.
- Recherche itérative : la population est améliorée génération après génération.
- Recherche fondée sur l'évaluation : seules les valeurs de fitness guident l'évolution.



II. Algorithme génétique canonique

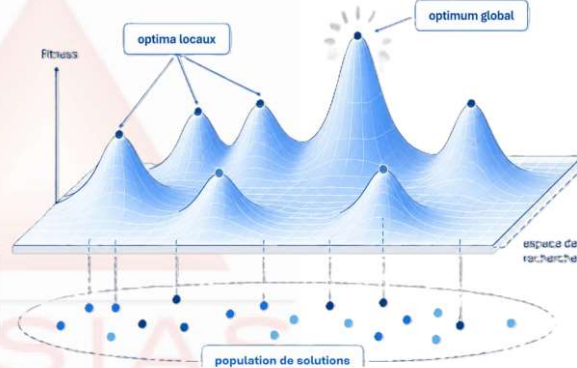
AG canonique

Formulation générale

Maximisation : $x^* = \arg \max F(x), x \in S$

- S : espace des solutions possibles.
- x : solution candidate.
- $F(x)$: fitness utilisée par l'algorithme.

Un AG est particulièrement utile lorsque l'espace S est discret, mixte, multimodal ou trop grand pour une exploration exhaustive.



II. Algorithme génétique canonique

AG canonique

Formulation générale

Notation	Signification	Rôle dans l'AG
S	Espace de recherche	Ensemble des solutions possibles
x_i^t	Individu i à la génération t	Une solution candidate
P_t	Population à la génération t	Ensemble des individus courants
$F(x)$	Fonction de fitness	Mesure de qualité / adaptation
p_c	Probabilité de croisement	Contrôle la recombinaison
p_m	Probabilité de mutation	Contrôle la variation aléatoire
T	Nombre maximal de générations	Critère d'arrêt courant

II. Algorithme génétique canonique

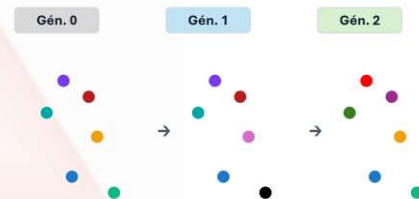
Population et génération

Population à la génération t

$$P_t = \{x_1^t, x_2^t, \dots, x_N^t\}$$

- N représente la taille de la population.
- Chaque x_i^t est une solution candidate.
- La génération t correspond à une itération de l'évolution.
- La population suivante P_{t+1} remplace totalement ou partiellement P_t .

Évolution de la population



II. Algorithme génétique canonique

Génotype et phénotype

Distinguer la représentation manipulée par l'algorithme et la solution réelle du problème



Exemple :

Génotype	Décodage	Phénotype
11000	binaire → décimal	$x = 24$
01101	binaire → décimal	$x = 13$

Point important

L'AG modifie le génotype. La fitness est souvent calculée après décodage, sur le phénotype.

II. Algorithme génétique canonique

Fonction de fitness

Définition formelle

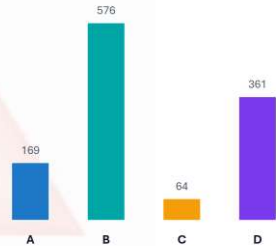
La fonction de fitness F associe à chaque individu x un score numérique qui mesure la qualité de la solution.

Si $F(x_1) > F(x_2)$, alors x_1 est préféré à x_2 .

Rôle dans l'algorithme

- Classer les individus.
- Calculer des probabilités de sélection.
- Mesurer les progrès générationnels.
- Définir un critère d'arrêt éventuel.

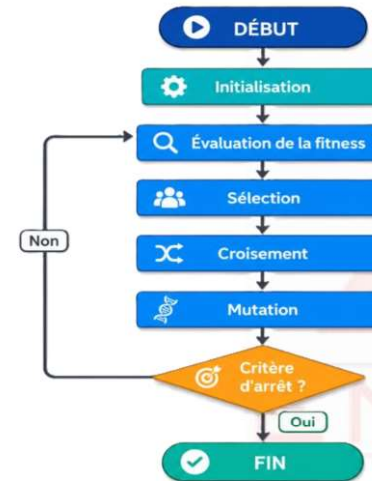
Exemple visuel : scores de fitness



La fitness est le lien entre le problème réel et le mécanisme d'évolution.

II. Algorithme génétique canonique

Cycle général d'un AG canonique



Lecture algorithmique

- Initialisation : créer P_0 .
- Évaluation : calculer $F(x)$ pour chaque individu.
- Sélection : choisir les parents selon leur qualité.
- Croisement : recombinaison de l'information génétique.
- Mutation : introduire de petites variations.
- Critère d'arrêt : décider si l'évolution continue.

Le cycle est répété jusqu'à atteindre un nombre maximal de générations, une fitness cible ou une stagnation prolongée.

II. Algorithme génétique canonique

Initialisation de la population

Objectif

Générer une population initiale P_0 suffisamment diversifiée pour couvrir plusieurs régions de l'espace de recherche.

Exemple : population binaire initiale



Une population initiale trop homogène peut limiter l'exploration des premières générations.

Stratégie	Principe	Avantage
Aléatoire	Tirage uniforme ou contrôlé	Diversité élevée
Heuristique	Utilise une connaissance métier	Démarrage plus prometteur
Hybride	Mélange aléatoire + heuristique	Compromis robuste

II. Algorithme génétique canonique

Evaluation

Attribuer une valeur de fitness à chaque individu

Principe

Pour chaque individu $x_i^t \in P_t$, on calcule la fitness $F(x_i^t)$. Cette valeur détermine son rang, sa probabilité de reproduction ou sa survie.

Évaluation populationnelle :

$$P_t = \{x_1^t, x_2^t, \dots, x_{N^t}^t\}$$

$$F(P_t) = \{F(x_1^t), F(x_2^t), \dots, F(x_{N^t}^t)\}$$

Individu	Chromosome	x	$F(x)=x^2$
A	01101	13	169
B	11000	24	576
C	01000	8	64
D	10011	19	361

Dans cet exemple, B est le plus adapté car sa fitness est maximale.

II. Algorithme génétique canonique

Sélection

Construire un pool de parents à partir des individus les plus prometteurs

Définition

La sélection choisit les individus autorisés à transmettre leur information génétique à la génération suivante.

- Les individus de meilleure fitness ont généralement une probabilité plus élevée d'être sélectionnés.
- Les sélections sont souvent réalisées avec remise : un même individu peut être choisi plusieurs fois.
- La pression de sélection contrôle l'équilibre entre exploitation et diversité.

Stratégie	Idée
Roulette	Probabilité proportionnelle à la fitness
Tournoi	Compétition entre k individus tirés au hasard
Rang	Probabilité basée sur le classement
SUS	Roulette à pointeurs régulièrement espacés

II. Algorithme génétique canonique

Croisement

Recombiner deux parents pour produire de nouveaux individus

Principe

Le croisement combine des segments ou des gènes de deux chromosomes parents. Il exploite les bonnes structures déjà présentes dans la population.



Rôle

- Créer de nouvelles combinaisons.
- Exploiter les informations utiles des parents.
- Produire une diversité structurée, moins aléatoire que la mutation.

Probabilité associée : $p_c (P_c)$
Si le croisement n'est pas appliqué, les parents peuvent être recopiés ou transmis selon la stratégie de remplacement.

II. Algorithme génétique canonique

Mutation

Introduire une variation locale pour maintenir la diversité

Principe

La mutation modifie aléatoirement un ou plusieurs gènes avec une probabilité généralement faible $p_m (P_m)$.

Rôles de la mutation

- Introduire de nouvelles valeurs de gènes.
- Éviter la convergence prématurée.
- Explorer des zones absentes de la population initiale.
- Réparer ou diversifier certaines solutions selon le codage.

Avant: 1 0 1 1 0 0 1

↓ gène muté

Après: 1 0 0 1 0 0 1

Attention :

Une mutation trop forte transforme l'AG en recherche quasi aléatoire ; une mutation trop faible peut réduire la diversité.

II. Algorithme génétique canonique

Remplacement

Construire la génération suivante P_{t+1}

Définition

Le remplacement détermine quels individus formeront la population de la génération suivante.



Stratégie	Principe
Générationnel	Les enfants remplacent toute l'ancienne population
Élitiste	Les meilleurs individus sont conservés
Steady-state	Quelques individus seulement sont remplacés

Élitisme

Mécanisme consistant à préserver directement un petit nombre des meilleurs individus afin d'éviter leur destruction par croisement ou mutation.

Le remplacement influence fortement la convergence et la diversité de la population.

II. Algorithme génétique canonique

Critères d'arrêt

Décider quand interrompre la boucle évolutionnaire

Critère	Principe
Nombre de générations	Arrêt après T générations
Fitness cible	Arrêt si $F(x^*)$ atteint un seuil
Stagnation	Arrêt si le meilleur score n'évolue plus
Temps de calcul	Arrêt après une limite de temps
Convergence	Arrêt si la population devient trop homogène

Exemple de règle combinée

Arrêter si $t = T$ ou si la meilleure fitness n'a pas progressé pendant K générations consécutives.

II. Algorithme génétique canonique

Paramètres de l'algorithme

Les hyperparamètres qui contrôlent la dynamique de recherche

Paramètre	Notation	Effet principal
Taille de population	N	Diversité et coût de calcul
Taux de croisement	p_c	Intensité de recombinaison
Taux de mutation	p_m	Diversité et exploration
Nombre de générations	T	Durée de l'évolution
Taux d'élitisme	e	Préservation des meilleurs
Méthode de sélection	—	Pression de sélection

Trade-off classique

- Augmenter N ou T améliore souvent l'exploration, mais augmente le coût de calcul.
- Augmenter p_m peut aider à sortir des optima locaux, mais peut aussi dégrader la convergence.

II. Algorithme génétique canonique

Pseudo-code de l'AG canonique

Entrées :

$F(x)$: fonction de fitness
N : taille de la population
 p_c : probabilité de croisement
 p_m : probabilité de mutation
T : nombre maximal de générations

Début

Initialiser aléatoirement une population P de N individus
Évaluer la fitness de chaque individu de P
 $t \leftarrow 0$

Tant que ($t < T$) **et** (critère d'arrêt non satisfait) **faire**

Sélectionner des parents à partir de P
Appliquer le croisement avec la probabilité p_c
Appliquer la mutation avec la probabilité p_m
Évaluer la fitness des nouveaux individus
Construire la nouvelle population P'
Remplacer P par P'
Mémoriser le meilleur individu trouvé
 $t \leftarrow t + 1$

Fin Tant que

Retourner le meilleur individu trouvé

Fin

- ✓ Le pseudo-code sépare la logique générale du problème traité.
- ✓ La fonction F est spécifique au problème.
- ✓ Les opérateurs dépendent du codage choisi.
- ✓ Le meilleur individu est généralement mémorisé pendant toute l'exécution.

III. Représentation et codage

Représentation

Pourquoi la représentation est-elle cruciale?

- ☐ La représentation relie le problème réel aux opérateurs génétiques
- ☐ Un AG ne manipule pas toujours directement la solution du problème.
- ☐ Un AG manipule une représentation codée : le chromosome.
- ☐ Le choix du codage influence la validité, la diversité et l'efficacité de recherche

Conséquences d'un mauvais codage

- ☐ solutions invalides après croisement ou mutation ;
- ☐ opérateurs peu informatifs ou destructifs ;
- ☐ fitness difficile à calculer ou coûteuse ;
- ☐ convergence prématurée ;
- ☐ recherche lente ou instable.

III. Représentation et codage

Représentation

Génotype, phénotype et fonctions de transformation

• Génotype

Représentation interne manipulée par l'algorithme génétique. Exemple : chaîne binaire, vecteur réel, permutation, arbre.

• Fonction de décodage

Transforme un chromosome en solution réelle exploitable :

$$\text{decode: } \mathcal{G} \rightarrow \mathcal{P}$$

où :

$\mathcal{G}$ = espace des génotypes

$\mathcal{P}$ = espace des phénotypes

• Phénotype

Solution réelle obtenue après décodage du génotype.

Exemple : valeur numérique, tournée TSP, emploi du temps, configuration de paramètres.



- Espace de génotypes: $\mathcal{G}$
- Espace de solutions: $\mathcal{P}$

- Décodage : $d : \mathcal{G} \rightarrow \mathcal{P}$
- Évaluation : $F(g) = f(d(g))$

III. Représentation et codage

Codage

Codage binaire : un chromosome est une chaîne de bits.

Le **codage binaire** représente une solution sous forme d'une chaîne de bits :

$$\text{chromosome} = b_1 b_2 \dots b_L, b_i \in \{0, 1\}$$

Chaque bit ou groupe de bits encode une **variable**, une **décision** ou une **caractéristique** de la solution.

Chromosome binaire :



Chaque position = un gène

Exemple sac-à-dos :

1 = objet sélectionné, 0 = objet non sélectionné

Cas d'utilisation

- Problèmes de décision **oui/non**.
- Sélection d'objets dans un sac-à-dos.
- Activation ou désactivation de composants.
- Représentation historique des premiers AG.
- Optimisation discrète ou combinatoire simple.

III. Représentation et codage

Codage

Codage binaire : un chromosome est une chaîne de bits.

Exemple

Problème

Maximiser $f(x)=x^2$ avec $x \in \{0, \dots, 31\}$.

Une solution x est codée par 5 bits.

Exemple : $g = 11000_2$

Décodage : $x = 24$

Fitness : $f(24)=576$

Population initiale

Individu	Chromosome	x	f(x)=x ²
A	01101	13	169
B	11000	24	576
C	01000	8	64
D	10011	19	361

III. Représentation et codage

Codage

Codage réel : Représentation naturelle pour l'optimisation continue

Le **codage réel** représente directement une solution par un vecteur de valeurs numériques continues :

$$\text{chromosome} = (x_1, x_2, \dots, x_n), x_i \in \mathbb{R}$$

Chaque gène correspond à une **variable réelle** ou à un **paramètre numérique** du problème.

Cas d'utilisation

- Optimisation de paramètres numériques.
- Réglage de modèles ou d'hyperparamètres.
- Optimisation de fonctions continues.
- Identification paramétrique.
- Conception d'ingénierie : dimensions, coefficients, poids, seuils.

III. Représentation et codage

Codage

Codage réel : Représentation naturelle pour l'optimisation continue

Exemple

Problème

On cherche à maximiser la fonction :
 $f(x_1, x_2) = 10 - (x_1 - 2)^2 - (x_2 + 1)^2$
avec :
 $x_1 \in [0; 5], x_2 \in [-3; 3]$

Représentation par codage réel

Chaque individu est représenté directement par un vecteur réel :
 $chromosome = [x_1 \ x_2]$

Individu	Chromosome réel	x_1	x_2	$f(x_1, x_2)$
A	[(1.50, -0.80)]	1.50	-0.80	9.71
B	[(3.20, -1.40)]	3.20	-1.40	8.40
C	[(0.70, 2.10)]	0.70	2.10	-1.30
D	[(2.10, -1.20)]	2.10	-1.20	9.95

III. Représentation et codage

Codage

Codage par permutation : Indispensable lorsque la solution est un ordre.

Exemple

Problème

On considère 5 villes :

A, B, C, D, E

Un chromosome peut représenter une tournée :

[A,C,E,B,D]

Position dans le chromosome	Élément	Signification
1	A	Ville visitée en premier
2	C	Ville visitée en deuxième
3	E	Ville visitée en troisième
4	B	Ville visitée en quatrième
5	D	Ville visitée en cinquième

III. Représentation et codage

Codage

Codage par permutation : Indispensable lorsque la solution est un ordre.

Le **codage par permutation** représente une solution sous forme d'un **ordre** ou d'une **séquence** d'éléments distincts :

$$chromosome = [e_1' e_2' \dots e_n']$$

où chaque élément apparaît **une seule fois**.

Ce codage est utilisé lorsque la qualité de la solution dépend principalement de l'**ordre des éléments**.

Cas d'utilisation

Le codage par permutation est adapté lorsque la solution dépend principalement d'un **ordre de passage**, d'un **classement** ou d'une **séquence d'exécution**.

- Problème du voyageur de commerce : ordre de visite des villes.
- Ordonnancement de tâches : ordre d'exécution des opérations.
- Routage de véhicules : ordre des clients à visiter.
- Planification de production : ordre de traitement des commandes.
- Affectation ordonnée : classement de ressources ou de priorités.

III. Représentation et codage

Codage

Codage par arbre : Représentation utilisée en programmation

La **représentation par arbre** encode une solution sous forme d'une structure hiérarchique composée de :

- **nœuds internes** : opérateurs, fonctions ou règles ;
- **feuilles** : variables, constantes ou actions élémentaires.

Elle est principalement utilisée en **programmation génétique**, où l'objectif est de faire évoluer automatiquement des expressions, règles ou programmes.

Forme générale

Un individu peut être représenté par un arbre :

$$\begin{array}{c} + \\ / \quad \backslash \\ \times \quad 3 \\ / \quad \backslash \\ x \quad y \end{array}$$

Cet arbre correspond à l'expression :

$$(x \times y) + 3$$

III. Représentation et codage

Codage

Codage par arbre : Représentation utilisée en programmation

La **représentation par arbre** encode une solution sous forme d'une structure hiérarchique composée de :

- **nœuds internes** : opérateurs, fonctions ou règles ;
- **feuilles** : variables, constantes ou actions élémentaires.

Elle est principalement utilisée en **programmation génétique**, où l'objectif est de faire évoluer automatiquement des expressions, règles ou programmes.

Cas d'utilisation

- Programmation génétique : Évolution automatique de programmes ou d'expressions symboliques.
- Régression symbolique: Recherche d'une formule mathématique expliquant des données.
- Classification par règles: Construction de règles de décision interprétables.
- Contrôle et jeux: Évolution de stratégies ou de comportements d'agents.
- Optimisation de politiques: Représentation de politiques sous forme de conditions et d'actions.

IV. Opérateurs génétiques

Rôle des opérateurs dans l'algorithme génétique

- ❑ Les **opérateurs génétiques** constituent le cœur du mécanisme évolutionnaire.
- ❑ Ils transforment une population courante P_t en une nouvelle population P_{t+1} .
- ❑ Ils permettent de répondre à quatre questions fondamentales :
 - **Quels individus sélectionner ?**
Identifier les solutions les plus prometteuses.
 - **Comment recombiner les solutions ?**
Mélanger l'information génétique des parents.
 - **Comment introduire de la nouveauté ?**
Modifier certains gènes pour maintenir la diversité.
 - **Comment construire la génération suivante ?**
Remplacer tout ou partie de la population.

III. Représentation et codage

Codage

Choisir la bonne représentation

Critère	Question à poser	Pourquoi c'est important
Validité	Les chromosomes décodent-ils des solutions acceptables ?	Évite les solutions impossibles
Localité	Petite modification du chromosome $\Rightarrow$ petite modification de la solution ?	Améliore la stabilité de la recherche
Expressivité	Toutes les solutions utiles peuvent-elles être représentées ?	Évite de restreindre inutilement l'espace
Redondance	Plusieurs chromosomes codent-ils la même solution ?	Peut ralentir ou biaiser la recherche
Compatibilité	Les opérateurs préservent-ils la structure ?	Évite des enfants incohérents
Coût	Le décodage et l'évaluation sont-ils coûteux ?	Impact direct sur le temps de calcul

IV. Opérateurs génétiques

Les opérateurs étudiés

- ❑ Dans cet axe, nous allons analyser les principaux mécanismes utilisés dans un algorithme génétique :

Opérateur	Question centrale	Rôle principal
Sélection	Qui se reproduit ?	Favoriser les bons individus
Croisement	Comment combiner deux parents ?	Exploiter les bonnes structures
Mutation	Comment créer de la diversité ?	Explorer de nouvelles solutions
Élitisme	Que faut-il préserver ?	Conserver les meilleures solutions
Remplacement	Comment former (P_{t+1}) ?	Construire la nouvelle génération

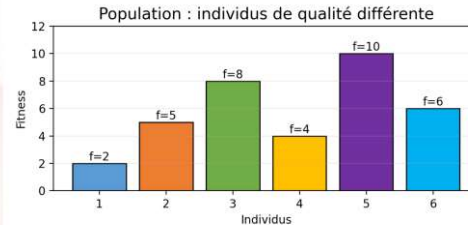


IV. Opérateurs génétiques

Sélection

Rôle de la sélection dans un AG

- ❑ Identifier les bonnes solutions dans la population courante.
- ❑ Favoriser leur reproduction afin qu'elles transmettent leurs caractéristiques.
- ❑ Réduire l'influence des mauvaises solutions sans nécessairement supprimer toute diversité.
- ❑ Orienter la recherche vers les régions prometteuses de l'espace de solutions.
- ❑ Construire un pool de parents qui sera ensuite utilisé par le croisement et la mutation.



Les individus de fitness élevée ont, en moyenne, davantage de chances d'être choisis.

IV. Opérateurs génétiques

Sélection

Sélection par roulette : Aussi appelée sélection proportionnelle à la fitness.

Principe

- ❑ La sélection par roulette choisit les individus avec une probabilité proportionnelle à leur fitness.
- ❑ Plus un individu est performant, plus il occupe une grande part de la roulette.
- ❑ Le tirage reste probabiliste : un individu faible peut encore être sélectionné.
- ❑ La sélection se fait généralement avec remise : un même individu peut être sélectionné plusieurs fois.

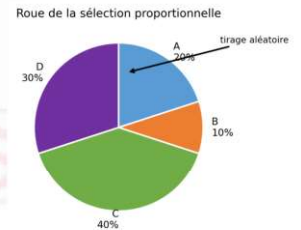
Pour une population de N individus :

$$p_i = \frac{F_i}{\sum_{j=1}^N F_j}$$

où :

- F_i : fitness de l'individu i ;
- p_i : probabilité de sélection de l'individu i .

Individu	Fitness	Probabilité
A	10	0,20
B	5	0,10
C	20	0,40
D	15	0,30



IV. Opérateurs génétiques

Sélection

Sélection par roulette : Aussi appelée sélection proportionnelle à la fitness.

Exemple

Individu	Fitness	Probabilité	Intervalle cumulatif
A	10	0,20	[0,00 ; 0,20[
B	5	0,10	[0,20 ; 0,30[
C	20	0,40	[0,30 ; 0,70[
D	15	0,30	[0,70 ; 1,00]

On tire r aléatoirement :

Si $r = 0,12$ alors A est sélectionné.

Si $r = 0,62$ alors C est sélectionné.

Si $r = 0,84$ alors D est sélectionné.

Construction des intervalles cumulatifs



IV. Opérateurs génétiques

Sélection

Sélection par tournoi : Des compétitions locales entre quelques individus tirés au hasard

Principe

- ❑ La sélection par tournoi consiste à choisir aléatoirement k individus de la population.
- ❑ Les individus sélectionnés sont comparés selon leur fitness.
- ❑ Le meilleur individu du groupe remporte le tournoi et il est placé dans le pool de parents.
- ❑ L'opération est répétée jusqu'à obtenir le nombre de parents souhaité.

Paramètre clé : taille du tournoi

k = nombre d'individus participant au tournoi

- Si k est petit : la pression de sélection est faible, la diversité est mieux conservée.
- Si k est grand : les meilleurs individus gagnent plus souvent, mais la diversité peut diminuer rapidement.

IV. Opérateurs génétiques

Sélection

Sélection par tournoi : Des compétitions locales entre quelques individus tirés au hasard

Principe

- ❑ La sélection par tournoi consiste à choisir aléatoirement **k** individus de la population.
- ❑ Les individus sélectionnés sont comparés selon leur fitness.
- ❑ Le meilleur individu du groupe remporte le tournoi et il est placé dans le pool de parents.
- ❑ L'opération est répétée jusqu'à obtenir le nombre de parents souhaité.

Paramètre clé : taille du tournoi

k = nombre d'individus participant au tournoi

- Si **k est petit** : la pression de sélection est faible, la diversité est mieux conservée.
- Si **k est grand** : les meilleurs individus gagnent plus souvent, mais la diversité peut diminuer rapidement.

Exemple

Pour un tournoi de taille $k=3$



IV. Opérateurs génétiques

Sélection

Sélection par rang :

Principe

- ❑ La sélection par rang ne sélectionne pas les individus directement selon leur valeur brute de fitness.
- ❑ Les individus sont d'abord classés du meilleur au moins bon.
- ❑ Une probabilité de sélection est ensuite attribuée selon le rang.
- ❑ Le meilleur individu reçoit la plus grande probabilité, mais les autres conservent une chance d'être sélectionnés.

On utilise ici une règle simple :

$$p_i = \frac{rang_i}{\sum rangs}$$

Exemple

Individu	Rang	Fitness	Probabilité
A	5	40	(5/15 = 0,33)
D	4	30	(4/15 = 0,27)
C	3	25	(3/15 = 0,20)
E	2	15	(2/15 = 0,13)
B	1	2	(1/15 = 0,07)

- L'individu A reste favorisé car il est le meilleur.
- L'individu B conserve une petite chance d'être sélectionné.
- Les écarts extrêmes de fitness sont atténués.
- La pression de sélection devient plus contrôlée.

IV. Opérateurs génétiques

Sélection

Sélection par rang :

Principe

- ❑ La sélection par rang ne sélectionne pas les individus directement selon leur valeur brute de fitness.
- ❑ Les individus sont d'abord classés du meilleur au moins bon.
- ❑ Une probabilité de sélection est ensuite attribuée selon le rang.
- ❑ Le meilleur individu reçoit la plus grande probabilité, mais les autres conservent une chance d'être sélectionnés.

On utilise ici une règle simple :

$$p_i = \frac{rang_i}{\sum rangs}$$

IV. Opérateurs génétiques

Sélection

Sélection SUS : Stochastic Universal Sampling

Principe

- ❑ La sélection SUS est une variante améliorée de la sélection par roulette.
- ❑ Au lieu de faire plusieurs tirages indépendants, SUS utilise un seul tirage initial puis plusieurs pointeurs régulièrement espacés.
- ❑ Les individus sont sélectionnés selon leur probabilité de fitness, mais avec une variance plus faible.
- ❑ Cette méthode produit un pool de parents plus stable et plus représentatif de la distribution des fitness.

Stochastic Universal Sampling : pointeurs régulièrement espacés



IV. Opérateurs génétiques

Sélection

Sélection SUS : Stochastic Universal Sampling

Exemple

Population initiale

Individu	Fitness	Probabilité
A	10	0,20
B	5	0,10
C	20	0,40
D	15	0,30

On veut sélectionner :

$$N = 4$$

parents.

Étape 1: Construire les intervalles cumulatifs

Individu	Probabilité	Intervalle cumulatif
A	0,20	[[0,00 ; 0,20[
B	0,10	[[0,20 ; 0,30[
C	0,40	[[0,30 ; 0,70[
D	0,30	[[0,70 ; 1,00[

IV. Opérateurs génétiques

Sélection

Sélection SUS : Stochastic Universal Sampling

Exemple

Population initiale

Individu	Fitness	Probabilité
A	10	0,20
B	5	0,10
C	20	0,40
D	15	0,30

On veut sélectionner :

$$N = 4$$

parents.

Étape 1: Construire les intervalles cumulatifs

Individu	Probabilité	Intervalle cumulatif
A	0,20	[[0,00 ; 0,20[
B	0,10	[[0,20 ; 0,30[
C	0,40	[[0,30 ; 0,70[
D	0,30	[[0,70 ; 1,00[

Étape 2 : Calculer l'espacement entre les pointeurs

Comme on veut sélectionner $N = 4$ parents :

$$d = \frac{1}{N} = \frac{1}{4} = 0,25$$

On choisit un premier nombre aléatoire :

$$r = 0,12$$

Les pointeurs sont alors :

$$\begin{aligned} r_1 &= 0,12 \\ r_2 &= 0,12 + 0,25 = 0,37 \\ r_3 &= 0,12 + 0,50 = 0,62 \\ r_4 &= 0,12 + 0,75 = 0,87 \end{aligned}$$

IV. Opérateurs génétiques

Sélection

Sélection SUS : Stochastic Universal Sampling

Exemple

Population initiale

Individu	Fitness	Probabilité
A	10	0,20
B	5	0,10
C	20	0,40
D	15	0,30

On veut sélectionner :

$$N = 4$$

parents.

Étape 1: Construire les intervalles cumulatifs

Individu	Probabilité	Intervalle cumulatif
A	0,20	[[0,00 ; 0,20[
B	0,10	[[0,20 ; 0,30[
C	0,40	[[0,30 ; 0,70[
D	0,30	[[0,70 ; 1,00[

Étape 2 : Calculer l'espacement entre les pointeurs

Comme on veut sélectionner $N = 4$ parents :

$$d = \frac{1}{N} = \frac{1}{4} = 0,25$$

On choisit un premier nombre aléatoire :

$$r = 0,12$$

Les pointeurs sont alors :

$$\begin{aligned} r_1 &= 0,12 \\ r_2 &= 0,12 + 0,25 = 0,37 \\ r_3 &= 0,12 + 0,50 = 0,62 \\ r_4 &= 0,12 + 0,75 = 0,87 \end{aligned}$$

Étape 3 : Associer chaque pointeur à un individu

Pointeur	Valeur	Intervalle	Individu sélectionné
r_1	0,12	[[0,00 ; 0,20[	A
r_2	0,37	[[0,30 ; 0,70[	C
r_3	0,62	[[0,30 ; 0,70[	C
r_4	0,87	[[0,70 ; 1,00[	D

Pool de parents obtenu :

$$[A, C, C, D]$$

IV. Opérateurs génétiques

Élitisme

Élitisme et stratégies de conservation

- ☐ L'élitisme consiste à conserver automatiquement un ou plusieurs des meilleurs individus d'une génération.
- ☐ Ces individus sont copiés directement dans la génération suivante, sans être modifiés par le croisement ou la mutation.
- ☐ L'objectif est d'éviter la perte accidentelle d'une très bonne solution.
- ☐ L'élitisme transforme l'AG en une recherche plus stable, car la meilleure fitness trouvée ne se dégrade pas d'une génération à l'autre.

Pourquoi utiliser l'élitisme ?

- Préserver les meilleures solutions déjà découvertes.
- Accélérer la convergence vers des régions prometteuses.
- Réduire l'effet du hasard dans la sélection, le croisement et la mutation.
- Garantir une amélioration monotone de la meilleure solution observée, si le meilleur individu est conservé.

IV. Opérateurs génétiques

Élitisme

Stratégies d'élitisme

Stratégie	Principe	Effet principal
Élitisme simple	Copier le meilleur individu vers P_{t+1}	Préserve la meilleure solution
Élitisme top-(k)	Copier les (k) meilleurs individus	Renforce l'exploitation
Élitisme proportionnel	Conserver un pourcentage de la population	Contrôle la pression de conservation
Élitisme avec remplacement	Les meilleurs remplacent les plus faibles	Améliore rapidement la population
Élitisme conditionnel	Conserver un individu seulement s'il améliore la génération précédente	Limite la domination excessive

Point de vigilance

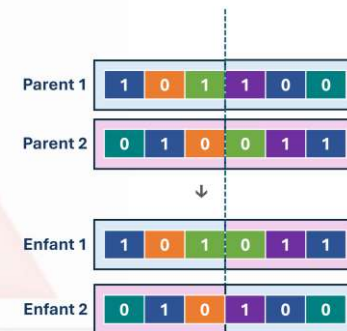
- Un élitisme trop faible peut laisser disparaître une bonne solution.
- Un élitisme trop fort peut réduire la diversité.
- Une population trop homogène risque de converger prématurément vers un optimum local.

IV. Opérateurs génétiques

Croisement

Rôle du croisement dans un AG

- Le croisement est l'opérateur qui combine deux individus parents pour produire un ou plusieurs enfants.
- Il intervient après la sélection, lorsque les individus les plus prometteurs ont été choisis pour la reproduction.
- Il permet de mélanger l'information génétique contenue dans les chromosomes parents.
- L'objectif est de créer de nouvelles solutions qui héritent de caractéristiques utiles provenant de plusieurs parents.
- Le croisement transforme le **pool de parents** en une population d'enfants candidats.
- Son efficacité dépend fortement du codage : binaire, réel ou permutation.
- Il doit être associé à la mutation pour maintenir une exploration suffisante.



Les enfants héritent chacun d'une partie de chaque parent.

IV. Opérateurs génétiques

Croisement

Panorama des principales stratégies de croisement

Chaque représentation impose des opérateurs compatibles avec la structure des solutions.

Famille	Codage visé	Opérateurs typiques	Idee principale
Segments	Binaire / symbolique	1 point, 2 points	Échanger des blocs contigus
Gène par gène	Binaire / symbolique	Uniforme	Choisir la source de chaque gène
Arithmétique	Réel	Moyenne, BLX- α , SBX	Interpoler entre deux vecteurs
Ordre	Permutation	OX, PMX, CX	Préserver une permutation valide

- un bon croisement doit produire des enfants valides et préserver les structures utiles du problème.
- le « meilleur » croisement dépend du problème et du codage.
- Un opérateur mal adapté peut produire des solutions invalides.

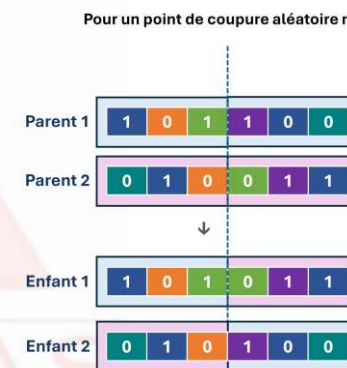
IV. Opérateurs génétiques

Croisement

Croisement à 1 point (single-point crossover)

Principe

- Choisir aléatoirement un point de coupure.
- Copier le segment gauche d'un parent.
- Compléter avec le segment droit de l'autre parent.
- Générer souvent deux enfants symétriques.
- Il est particulièrement adapté aux chromosomes de type binaire, entier ou symbolique de longueur fixe.



Les enfants héritent chacun d'une partie de chaque parent.

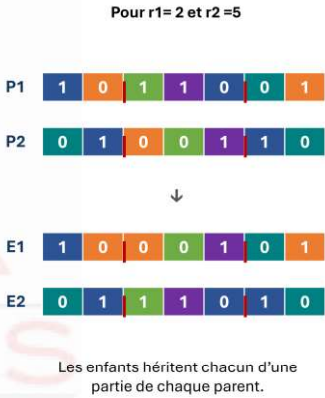
IV. Opérateurs génétiques

Croisement

Croisement à 2 points (two-point crossover)

Principe

- ❑ Choisir deux positions de coupure.
- ❑ Échanger le segment situé entre les deux coupures.
- ❑ Préserve davantage les blocs à gauche et à droite.
- ❑ Souvent plus diversifiant que le croisement à un point.



IV. Opérateurs génétiques

Croisement

Croisement arithmétique (codage réel)

Principe

- Représenter chaque individu par un vecteur de valeurs réelles.
- Choisir deux parents P_1 et P_2 .
- Définir un coefficient de pondération :
 $\alpha \in [0, 1]$
- Combiner les deux parents par interpolation linéaire.
- Produire un ou deux enfants situés entre les deux parents dans l'espace de recherche.
- Ce croisement est plus naturel pour l'optimisation continue que pour les chaînes binaires.
- Variantes connues : **BLX- α** , **SBX** (simulated binary crossover).

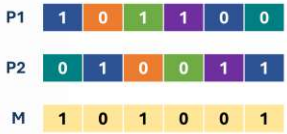
IV. Opérateurs génétiques

Croisement

Croisement uniforme (uniform crossover)

Principe

- ❑ Associer chaque position du chromosome à une décision indépendante.
- ❑ Générer un masque binaire aléatoire de même longueur que les parents.
- ❑ Choisir le gène du premier parent lorsque la valeur du masque vaut 1.
- ❑ Choisir le gène du second parent lorsque la valeur du masque vaut 0.
- ❑ Construire un enfant en combinant les gènes sélectionnés position par position.
- ❑ Former éventuellement un second enfant en inversant le rôle des parents.



Position	1	2	3	4	5	6
Masque	1	0	1	0	0	1
Enfant E1	P1	P2	P1	P2	P2	P1



IV. Opérateurs génétiques

Croisement

Croisement arithmétique (codage réel)

Principe

- Représenter chaque individu par un vecteur de valeurs réelles.
- Choisir deux parents P_1 et P_2 .
- Définir un coefficient de pondération :
 $\alpha \in [0, 1]$
- Combiner les deux parents par interpolation linéaire.
- Produire un ou deux enfants situés entre les deux parents dans l'espace de recherche.
- Ce croisement est plus naturel pour l'optimisation continue que pour les chaînes binaires.
- Variantes connues : **BLX- α** , **SBX** (simulated binary crossover).

Formule générale

Pour deux parents réels :

$$P_1 = (x_1, x_2, \dots, x_n)$$

$$P_2 = (y_1, y_2, \dots, y_n)$$

Un enfant peut être calculé par :

$$E_1 = \alpha P_1 + (1 - \alpha) P_2$$

Un second enfant peut être obtenu par :

$$E_2 = (1 - \alpha) P_1 + \alpha P_2$$

Exemple numérique

$$P_1 = [2.0, 5.0, 1.0, 4.0]$$

$$P_2 = [6.0, 1.0, 3.0, 2.0]$$

$$E_1 = 0,3P_1 + 0,7P_2 = [4,8 ; 2,2 ; 2,4 ; 2,6]$$

Avec $\alpha = 0,3$

$$E_2 = 0,7P_1 + 0,3P_2 = [3,2 ; 3,8 ; 1,6 ; 3,4]$$

IV. Opérateurs génétiques

Croisement

Croisement d'ordre (OX) pour les permutations

Principe

- Utiliser le croisement OX lorsque le chromosome représente un ordre.
- Choisir deux points de coupure dans les deux parents.
- Copier le segment compris entre les deux points depuis le premier parent.
- Parcourir le second parent dans son ordre naturel.
- Compléter les positions vides avec les éléments absents, sans duplication.
- Préserver l'ordre relatif des éléments provenant du second parent.

Exemple:

P1 = [1 2 | 3 4 5 | 6 7 8]
 P2 = [4 6 | 8 7 2 | 1 3 5]
 1) Copier le segment central de P1 :
 [_ _ | 3 4 5 | _ _ _]
 2) Lire P2 en ignorant {3,4,5} :
 6, 8, 7, 2, 1
 3) Compléter les cases restantes :
 Enfant = [6 8 | 3 4 5 | 7 2 1]

IV. Opérateurs génétiques

Croisement

Croisement PMX (Partially Mapped Crossover)

Principe

- Utiliser PMX lorsque le chromosome représente une permutation.
- Choisir deux points de coupure communs aux deux parents.
- Copier le segment central d'un parent dans l'enfant.
- Établir une table de correspondance entre les éléments échangés.
- Compléter les positions restantes avec les gènes de l'autre parent.
- Résoudre les conflits à l'aide de la table de correspondance.
- Garantir une permutation valide : aucun doublon, aucun élément manquant.

Exemple:

P1 = [1 2 | 3 4 5 | 6 7 8]
 P2 = [4 6 | 8 7 2 | 1 3 5]
 Segment copié depuis P2 :
 [_ _ | 8 7 2 | _ _ _]
Correspondances : 3 ↔ 8, 4 ↔ 7, 5 ↔ 2
 • Compléter avec P1 en corrigeant les doublons.
 • 2 déjà présent → 2 ↔ 5 → placer 5.
 • 7 déjà présent → 7 ↔ 4 → placer 4.
 • Enfant final : [1 5 | 8 7 2 | 6 4 3]

IV. Opérateurs génétiques

Mutation

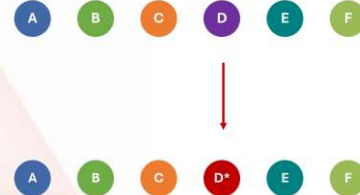
Rôle de la mutation dans un AG

Définition

La **mutation** est un opérateur génétique qui modifie aléatoirement un ou plusieurs gènes d'un chromosome avec une faible probabilité p_m .

Rôle

- Introduire de la nouveauté dans les chromosomes.
- Maintenir la diversité génétique au fil des générations.
- Éviter la convergence prématurée vers un optimum local.
- Réactiver l'exploration lorsque la population devient trop homogène.
- Corriger indirectement certaines pertes d'information causées par la sélection ou le croisement.
- Explorer de nouvelles régions de l'espace de recherche.



Un individu peut acquérir une variation nouvelle qui n'existait pas dans les parents.

IV. Opérateurs génétiques

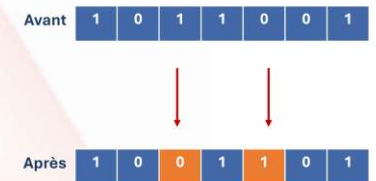
Mutation

Principe de la mutation

Formalisme minimal

Pour chaque gène g_i :
 si $r < p_m$ alors muter(g_i)

- r est un nombre aléatoire dans $[0, 1]$.
- p_m est le taux de mutation.
- La modification dépend du codage : binaire, réel, permutation, arbre.



Les positions 3 et 5 ont muté : la variation est locale mais il peut changer la trajectoire de recherche.

IV. Opérateurs génétiques

Mutation

Mutation binaire

Principe :

- **Sélectionner** un ou plusieurs bits d'un chromosome selon une probabilité de mutation p_m .
- **Inverser** la valeur du bit choisi :
 $0 \rightarrow 1$
 $1 \rightarrow 0$
- **Produire** un nouveau chromosome légèrement différent du chromosome initial.

$p_m = 0,10$; les bits 3 et 5 sont sélectionnés pour mutation

Avant

1	0	1	1	0	0	1
---	---	---	---	---	---	---



Après

1	0	0	1	1	0	1
---	---	---	---	---	---	---

Position	1	2	3	4	5	6	7
Action	—	—	1→0	—	0→1	—	—

IV. Opérateurs génétiques

Mutation

Mutation réelle

Principe :

- La mutation réelle s'applique lorsque le chromosome est représenté par un vecteur de valeurs numériques continues :

$$C = (x_1, x_2, \dots, x_n)$$

- Elle consiste à modifier légèrement un ou plusieurs gènes réels afin de produire une nouvelle solution.
- Pour un gène réel x_i , la mutation peut s'écrire :

$$x'_i = x_i + \epsilon$$

où :

- x_i est la valeur initiale du gène ;
- ϵ est une petite perturbation aléatoire ;
- x'_i est la nouvelle valeur après mutation.

IV. Opérateurs génétiques

Mutation

Mutation réelle

Principe :

- La mutation réelle s'applique lorsque le chromosome est représenté par un vecteur de valeurs numériques continues :
- $$C = (x_1, x_2, \dots, x_n)$$
- Elle consiste à modifier légèrement un ou plusieurs gènes réels afin de produire une nouvelle solution.
 - Pour un gène réel x_i , la mutation peut s'écrire :
- $$x'_i = x_i + \epsilon$$

où :

- x_i est la valeur initiale du gène ;
- ϵ est une petite perturbation aléatoire ;
- x'_i est la nouvelle valeur après mutation.

Types courants de mutation réelle

Type de mutation	Principe
Mutation uniforme	Remplacer un gène par une valeur aléatoire dans son intervalle autorisé
Mutation gaussienne	Ajouter un bruit aléatoire suivant une loi normale
Mutation non uniforme	Réduire progressivement l'amplitude de mutation au fil des générations

IV. Opérateurs génétiques

Mutation

Mutation réelle

Principe :

- La mutation réelle s'applique lorsque le chromosome est représenté par un vecteur de valeurs numériques continues :

$$C = (x_1, x_2, \dots, x_n)$$

- Elle consiste à modifier légèrement un ou plusieurs gènes réels afin de produire une nouvelle solution.
- Pour un gène réel x_i , la mutation peut s'écrire :

$$x'_i = x_i + \epsilon$$

où :

- x_i est la valeur initiale du gène ;
- ϵ est une petite perturbation aléatoire ;
- x'_i est la nouvelle valeur après mutation.

Types courants de mutation réelle

Type de mutation	Principe
Mutation uniforme	Remplacer un gène par une valeur aléatoire dans son intervalle autorisé
Mutation gaussienne	Ajouter un bruit aléatoire suivant une loi normale
Mutation non uniforme	Réduire progressivement l'amplitude de mutation au fil des générations

Exemple

Étape	Chromosome réel
Avant	[2.18 ; -1.04 ; 0.75]
Mutation	$\epsilon = +0.10$ sur le 2e gène
Après	[2.18 ; -0.94 ; 0.75]

IV. Opérateurs génétiques

Mutation

Mutation dans les permutations

Principe :

La mutation dans les permutations s'applique lorsque le chromosome représente un ordre d'éléments :

$$C=[e_1,e_2,\dots,e_n]$$

L'objectif est de modifier légèrement cet ordre tout en conservant une permutation valide :

- aucun élément ne doit être dupliqué ;

- aucun élément ne doit disparaître ;
- tous les éléments doivent rester présents une seule fois.

Stratégies courantes

Mutation	Principe	Exemple
Swap	Échanger deux positions	[A,B,C,D,E] → [A,D,C,B,E]
Insertion	Retirer un élément et l'insérer ailleurs	[A,B,C,D,E] → [A,C,D,B,E]
Inversion	Inverser un segment	[A,B,C,D,E] → [A,D,C,B,E]
Scramble	Mélanger aléatoirement un sous-segment	[A,B,C,D,E] → [A,D,B,C,E]

IV. Opérateurs génétiques

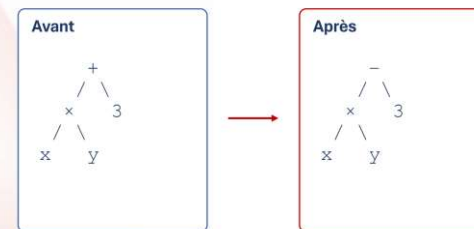
Mutation

Mutation par arbre

Principe :

- ❑ La mutation par arbre est utilisée lorsque l'individu est représenté par une structure arborescente, comme en programmation génétique.
- ❑ Elle consiste à modifier une partie de l'arbre afin de produire une nouvelle expression, règle ou stratégie.
- ❑ Contrôler la profondeur maximale pour éviter le bloat.

Exemple visuel : mutation de nœud



Le nœud racine "+" est remplacé par "-".

ENSIAS

IV. Opérateurs génétiques

Mutation

Mutation par arbre

Types courants de mutation

Type de mutation	Principe	Exemple
Mutation de feuille	Remplacer une variable ou une constante	$x \rightarrow 2$
Mutation de fonction	Remplacer un opérateur par un autre opérateur compatible	$+ \rightarrow *$
Mutation de sous-arbre	Remplacer une branche entière par un nouveau sous-arbre	$(x+1) \rightarrow (x * y)$
Mutation par croissance	Ajouter une nouvelle branche à un arbre existant	$x \rightarrow (x+1)$

ENSIAS

IV. Opérateurs génétiques

Remplacement

Définition

- ❑ Le **remplacement** est l'étape qui décide **quels individus formeront la nouvelle population** P_{t+1} après la sélection, le croisement et la mutation.
- ❑ Autrement dit, après avoir créé des enfants, il faut répondre à la question :
« Parmi les anciens individus, les parents et les nouveaux enfants, lesquels garder pour la génération suivante ? »

Rôle du remplacement

- ❑ Dans un algorithme génétique, on part d'une population courante : P_t
- ❑ Puis on génère de nouveaux individus par sélection, croisement et mutation. Le remplacement construit alors : P_{t+1}
- ❑ Il permet de :
 - garder la taille de population constante ;
 - intégrer les nouveaux enfants dans la population ;
 - supprimer certains anciens individus ;
 - préserver éventuellement les meilleurs individus ;
 - contrôler la diversité et la vitesse de convergence.

ENSIAS

IV. Opérateurs génétiques

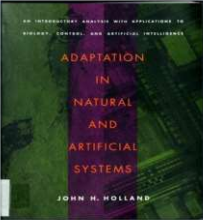
Remplacement

Principales stratégies de remplacement

Stratégie	Principe
Remplacement générationnel	Toute l'ancienne population est remplacée par les enfants.
Remplacement élitiste	Les meilleurs individus de P_t sont conservés dans P_{t+1} .
Remplacement steady-state	Quelques enfants remplacent seulement quelques mauvais individus.
Remplacement par compétition	Un enfant remplace un parent seulement s'il est meilleur.
Remplacement aléatoire contrôlé	Certains individus sont remplacés aléatoirement pour maintenir la diversité.

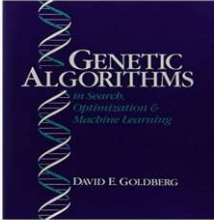
- ☐ Le remplacement contrôle la transition entre deux générations : il détermine quelles solutions survivent, quelles nouvelles solutions entrent dans la population, et comment l'algorithme équilibre amélioration et diversité.

Références pour approfondir



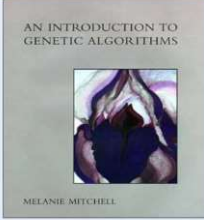
Holland, J. H. (1975/1992)
Adaptation in Natural and Artificial Systems

Fondements théoriques des systèmes adaptatifs et des AG.



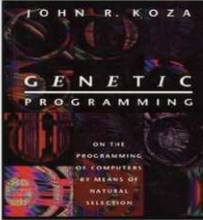
Goldberg, D. E. (1989)
Genetic Algorithms in Search, Optimization, and Machine Learning

Référence pédagogique classique : sélection, schémas, applications.



Mitchell, M. (1998)
An Introduction to Genetic Algorithms

Introduction claire : concepts, limites et études de cas.



Koza, J. R. (1992)
Genetic Programming

Extension des AG à l'évolution automatique de programmes.

À privilégier : Goldberg pour l'intuition et les mécanismes ; Holland pour les bases théoriques ; Mitchell pour l'introduction structurée ; Koza pour la programmation génétique.

IV. Opérateurs génétiques

Mutation

Mutation par arbre

Types courants de mutation

Type de mutation	Principe	Exemple
Mutation de feuille	Remplacer une variable ou une constante	$x \rightarrow 2$
Mutation de fonction	Remplacer un opérateur par un autre opérateur compatible	$+ \rightarrow *$
Mutation de sous-arbre	Remplacer une branche entière par un nouveau sous-arbre	$(x+1) \rightarrow (x * y)$
Mutation par croissance	Ajouter une nouvelle branche à un arbre existant	$x \rightarrow (x+1)$

Références pour approfondir

Ressources web

DEAP Documentation
DEAP Documentation

PyGAD Documentation
PyGAD Documentation

pymoo — NSGA-II Documentation
pymoo — NSGA-II Documentation